Plant Guardian

Four-member team study guide

A technical presentation reference for explaining the product, individual contributions, architecture, GCP services, and end-to-end data flow.

Core message

Most trackers tell you good or bad. Plant Guardian tells you how urgent.

Project stack: Next.js + TypeScript + Tailwind CSS | FastAPI + Python + SQLAlchemy | PostgreSQL | Docker | Google Cloud

1. Project overview

Plant Guardian is a full-stack plant-care dashboard for houseplant owners. Each plant has a private profile, watering schedule, last-watered timestamp, notes, location, sunlight requirement, pet-safety context, and a backend-calculated urgency state. The dashboard turns raw dates into clear actions: Healthy, Needs Water Soon, or Overdue / High Risk.

The application supports CRUD operations, room filtering, search, sorting, watering history, growth XP, streaks, seasonal adjustments, browser push reminders, Vacation Mode caretaker briefings, Google sign-in, and Gmail sharing.

High-level architecture

Layer	Technology	Responsibility
Browser	Next.js, React, TypeScript, Tailwind	Dashboard, forms, cards, animations, accessibility
API	FastAPI, Pydantic	Validation, REST endpoints, authentication, orchestration
Business services	Python service modules	Risk, seasons, streaks, pet safety, notifications
Persistence	SQLAlchemy + PostgreSQL	Users, plants, watering history, subscriptions
Cloud	Cloud Run and supporting GCP services	Containers, secrets, database, builds, scheduling, events

Data flow

A user interacts with the Next.js dashboard. The frontend calls the FastAPI REST API. FastAPI validates the request with Pydantic, reads or writes PostgreSQL through SQLAlchemy, and invokes isolated business services. Computed metrics are returned in the response rather than stored as stale values. For reminders, Cloud Scheduler calls the protected dispatch endpoint; the backend publishes Pub/Sub events or sends Web Push directly, depending on configuration.

2. Member 1 - Frontend and User Experience

Primary responsibility: turn plant-care data into a calm, understandable, responsive product interface.

Core technologies

• Next.js App Router provides the application shell and production rendering.

• React components hold UI state for forms, dialogs, filters, toasts, reminders, and watering actions.

• TypeScript types mirror backend response contracts so new fields are used safely.

• Tailwind CSS and CSS design tokens provide responsive layouts, status colors, focus states, and the light/dark green visual system.

• Phosphor icons and inline SVG are used for interface symbols and the Gmail mark without adding image dependencies.

Owned features

• Dashboard summary cards for total plants, Healthy, Needs Water Soon, Overdue, and average on-time percentage.

• Plant cards with nickname, species, room, sunlight, health badge, risk score, watering schedule, notes, edit/delete controls, and Just Watered.

• Risk-score progress visualization and color states: green, amber, and red.

• Search by nickname or species; dynamic room filters; pet-safety filters; sorting by risk, recent watering, or name.

• Add and edit forms with labels, date/time controls, numeric validation, and understandable error messages.

• Plant growth avatar, XP bar, watering history grid, streak indicators, floating XP feedback, and milestone toasts.

• Vacation Mode dialog, caretaker briefing display, copy action, Gmail compose redirect, pet warning, and seasonal adjustment notice.

• Notification settings UI with browser permission handling, enable/save/test/disable states, loading skeletons, and error feedback.

Frontend talking points

The frontend does not reimplement core risk math. It displays the backend's risk_score, status, days_until_due, seasonal frequency, streak, XP, and pet-safety fields. This keeps the API authoritative and prevents the UI from drifting from server behavior. Components are reusable and communicate through typed props and service functions.

3. Member 2 - Backend API and Plant-Care Logic

Primary responsibility: expose stable REST contracts and keep all care decisions in testable backend services.

Core technologies

• FastAPI defines route handlers, dependency injection, status codes, and exception responses.

• Pydantic schemas validate incoming plant, user, vacation, notification, and authentication payloads.

• SQLAlchemy provides database models, relationships, queries, and transaction handling.

• Pure Python services isolate calculations from HTTP and database code, making them unit-testable.

Owned features and concepts

• Plant REST API: GET collection, GET by ID, POST, PUT/PATCH, DELETE, and POST /water.

• Risk formula: risk_score = min(100, (days_since_watered / effective_frequency) * 100), rounded for display.

• Status thresholds: 0-39 Healthy; 40-69 Needs Water Soon; 70-100 Overdue / High Risk.

• Days calculation: days_since_watered = current date - last_watered; days_until_due = frequency - days_since_watered.

• Season service adjusts frequency by India-oriented factors: Summer 0.75, Monsoon 1.25, Post-monsoon 1.0, Winter 1.4.

• Streak service calculates garden-level consistency, current and longest streaks, history dots, XP, growth stage, and mood.

• Pet-safety service resolves species from a curated static dataset and composes placement guidance from toxicity and sunlight.

• Vacation service receives structured plant care data and calls the separate Vacation Mode service without coupling route code to prompt details.

• Notification service dispatches only when a plant is Needs Water Soon or Overdue, with daily idempotency and safe cancellation if a plant was watered before delivery.

Backend talking points

Calculated values such as risk and seasonal frequency are computed per request. Stored data contains user-entered facts and history; derived values are not persisted, preventing stale urgency states. Route logic stays thin: authenticate, validate, call a service, and serialize the response.

4. Member 3 - Database, Authentication, and Notifications

Primary responsibility: make the garden private, durable, and capable of delivering timely care reminders.

Database model

Entity	Important fields	Purpose
User	email, password_hash, full_name, place, pets, timezone	Private profile and ownership boundary

Entity	Important fields	Purpose
Plant	nickname, species, room, sunlight, frequency, last_watered, notes, xp	Current plant record and gamification state
Watering	plant_id, watered_at	Append-only care history used for streaks and consistency
PushSubscription	endpoint, keys, timezone, reminder_time, enabled	Browser push delivery configuration
NotificationDelivery	subscription_id, plant_id, kind, care_date, status	Idempotency, retries, and delivery tracking

Owned features

• Signup and login with secure password hashing, session cookies, expiration, and per-user plant isolation.

• Google sign-in using Google Identity Services and a backend token-verification path.

• Profile creation requiring name, place, and household pet details.

• Alembic migrations for schema evolution while preserving existing rows.

• Watering history insertion when Just Watered is used; plant deletion cascades to watering records.

• Push permission flow in the browser and subscription persistence in PostgreSQL.

• Reminder dispatch deduplicates by plant, subscription, notification kind, and local care date.

• Notification messages explain the action: water soon within the remaining window, or water an overdue plant.

Authentication and notification talking points

Authentication establishes who owns the garden. Every plant query is scoped to the authenticated user. Browser notifications require permission and VAPID configuration. Cloud Scheduler initiates scans, while Pub/Sub can queue delivery for resilient event-driven processing. If a queued plant is watered before delivery, the worker cancels the stale notification.

5. Member 4 - Cloud, DevOps, and Quality

Primary responsibility: make the local product deployable, repeatable, secure, and verifiable in Google Cloud.

GCP services and their roles

GCP service	How Plant Guardian uses it
Cloud Run	Runs frontend, backend, Vacation Mode, and AI Assistant containers as stateless services.
Cloud SQL	Managed PostgreSQL for users, plants, watering history, and notification records.
Secret Manager	Stores Groq and VAPID private keys; secrets are injected at runtime instead of committed.
Artifact Registry	Private Docker image registry for service images.
Cloud Build	Builds container images from source and publishes them to Artifact Registry.
IAM and service accounts	Separate runtime identities with least-privilege roles for Cloud SQL, secrets, Pub/Sub, and invocation.
Cloud Scheduler	Calls the protected notification dispatch endpoint every 15 minutes.
Pub/Sub	Queues notification delivery events so push delivery can be asynchronous and retried.

DevOps ownership

• Dockerfiles package each service; Docker Compose runs the local multi-service environment.

• Environment variables keep database URLs, API URLs, OAuth IDs, and secrets out of source code.

• The deployment script enables APIs, creates or reuses infrastructure, configures IAM, builds images, deploys services, and configures Scheduler and Pub/Sub.

• Cloud Run service URLs are wired into frontend build substitutions and service environment variables.

• The deployment workflow supports later GCP migration because the backend is stateless and data access is separated from routes.

Quality ownership

• Backend pytest covers API behavior, risk, streaks, pet safety, authentication, notifications, and cascade deletion.

• Frontend Vitest covers service functions, query behavior, notification helpers, and avatar behavior.

• ESLint catches TypeScript and React issues; Next.js production build catches type and bundling problems.

• Docker Compose configuration and deployment script syntax are checked before release.

• Manual smoke tests verify signup, adding a plant, watering, vacation planning, reminders, and GCP URLs.

6. End-to-end feature walkthroughs

Adding and watering a plant

1. Member 1's form collects nickname, species, room, sunlight, frequency, last watered, and notes. 2. Member 2's endpoint validates the payload and resolves pet safety. 3. Member 3's database layer stores the plant under the authenticated user. 4. The response returns risk and care metrics. 5. Clicking Just Watered creates a Watering record, recalculates metrics and XP, and updates the card without a page reload.

Reminder flow

1. The user grants browser notification permission and saves a subscription. 2. Cloud Scheduler calls the authenticated dispatch route every 15 minutes. 3. The backend calculates each plant's current status. 4. Healthy plants are skipped; Needs Water Soon and Overdue plants create one daily delivery. 5. Pub/Sub queues an event when enabled. 6. The worker verifies the queued delivery is still relevant and sends Web Push. 7. The browser service worker displays the notification and opens the dashboard when clicked.

Vacation Mode flow

The user chooses any future departure date and a later return date, selects plants, and generates a briefing. The frontend sends structured care data to Vacation Mode. The service returns a watering schedule and caretaker message, including seasonal and pet-handling context. The user can copy the message or open Gmail compose to share it manually.

7. Presentation questions and model answers

Question	Answer
What is unique about Plant Guardian?	It converts watering history into an urgency score and action state, then combines that with seasonal schedules, pet safety, reminders, and a motivating growth system.
Why calculate risk on the backend?	The backend is the single source of truth. Every client receives consistent calculations and no stale stored risk values exist.
Why PostgreSQL?	It provides durable relational storage, constraints, relationships, and a straightforward path from local development to Cloud SQL.
Why separate services?	The API owns data and care logic, while Vacation Mode and AI Assistant can scale or change independently behind stable service boundaries.
How is security handled?	Sessions and password hashing protect users, CORS is configured, inputs are validated, secrets use environment variables and Secret Manager, and Cloud Run callers use IAM/OIDC.
How does the app scale later?	Cloud Run services are stateless, images are containerized, and PostgreSQL is externalized to Cloud SQL. This supports independent scaling without rewriting business logic.
What does the team demonstrate?	Signup, empty private garden, adding plants, filtering/search, risk states, watering update, growth avatar, reminders, Vacation Mode, Gmail sharing, and deployed GCP services.

8. Local and cloud verification checklist

• Start local services with docker compose up --build.

• Create a new account and confirm the initial plant list is empty and private.

• Add plants with different frequencies and dates; verify all three urgency states.

• Use Just Watered and confirm risk, status, streak/XP, and avatar update without reload.

• Open Vacation Mode and enter a future departure date plus a later return date.

• Enable reminders, use Test for permission/delivery validation, and verify soon/overdue copy.

• Run backend: cd backend && python -m pytest -q.

• Run frontend: cd frontend && npm test && npm run lint && npm run build.

• Deploy with bash deploy/gcp-deploy.sh after configuring .env and authenticated gcloud.

• Verify Cloud Run URLs, Cloud Scheduler job, Pub/Sub subscription, Cloud SQL connectivity, and Secret Manager bindings.

9. One-minute team introduction

Member 1 built the product experience users see. Member 2 built the API and plant-care intelligence. Member 3 made accounts, data persistence, authentication, and reminders reliable. Member 4 containerized, tested, secured, and deployed the system on GCP. Together, the four layers turn a simple watering tracker into a production-ready care dashboard.